

Section A – Vectors

Problem 1: Recording Volunteer Names

Background: During the recruitment phase of a breast cancer screening study, the first three volunteers to register were *Hashem*, *Haris*, and *Mostafa*. Researchers often need to keep simple participant lists before linking them to detailed patient IDs. A vector is a good way to manage this information quickly in R.

Task: Create a vector with their names and print the first volunteer.

Problem 2: Assigning Patient IDs

Background: Each patient in a clinical trial is assigned a numeric ID instead of using their actual name for confidentiality. In your hypertension drug trial, the first four patients have IDs 1, 2, 3, 4. You want to store them for later reference.

Task: Store these IDs in a numeric vector and print them.

Problem 3: Student Participants in a Workshop

Background: In a medical statistics workshop, three medical students — *Maria*, *Shirmin*, and *Alam* — participated in the hands-on R training session. Before analyzing their performance, you want to check how R classifies this vector of names (text or numeric?).

Task: Create a vector of these names and check its class.

Problem 4: Extracting Test Results

Background: A cardiologist recorded a patient’s systolic blood pressure at three consecutive visits: 120, 132, 138. Since blood pressure data is collected longitudinally, researchers may want to extract specific visits for comparison.

Task: Store the readings in a vector and extract the last two readings.

Problem 5: Labeling Gene Expression

Background: In a small pilot study, you measured the expression counts (raw RNA-seq reads) for three cancer-related genes: *TP53* (230), *BRCA1* (120), and *MYC* (450). Properly labeling gene expression values helps prevent confusion during analysis.

Task: Store the values in a vector, assign the gene names as labels, and extract *BRCA1*’s value.

Section B – Matrices

Problem 6: Creating an Expression Matrix

Background: You measured expression values for 4 genes across 3 patients. In biomedical research, such rectangular data (genes × patients) is often stored as a matrix.

Task: Create a 4×3 matrix using values 1:12 filled row by row.

Problem 7: Checking Data Structure

Background: Sometimes researchers mistakenly store gene expression data as a “list” or “data frame” instead of a “matrix.” Before applying matrix-specific functions, you

must confirm its type.

Task: Use `class()` on the matrix from Problem 6 to check.

Problem 8: Mixing Data Types

Background: In a lab notebook digitization project, someone mistakenly entered "hello", "hi", "bye" among numeric gene expression values. R will automatically coerce all entries into one type, which can distort the dataset.

Task: Create a 2x3 matrix mixing text and numbers. Observe what happens.

Problem 9: Combining Patient Results into a Matrix

Background: You measured glucose levels at three time points (fasting, 1-hour, 2-hour after glucose drink) for three patients:

- Asif: 90, 160, 120
- Harun: 100, 180, 150
- Afridi: 80, 145, 110

Such data is best represented as a matrix for cross-patient comparisons.

Task: Create a matrix from these three patient vectors.

Problem 10: Adding Column Names for Time Points

Background: The three columns correspond to the three test conditions: "Baseline", "Treatment1", "Treatment2". Clear labels help make data interpretable at a glance.

Task: Assign these labels as column names.

Problem 11: Adding Row Names for Patients

Background: You want the rows of your glucose matrix labeled with patient names for clarity. Without labels, values could easily be misattributed.

Task: Assign "Asif", "Harun", and "Afridi" as row names.

Problem 12: Extracting a Specific Value

Background: A physician wants to know *Asif's* glucose level during "Treatment2". Rather than scanning the entire table, you can extract it directly by name.

Task: Extract this value using row and column labels.

Problem 13: Comparing Two Patients

Background: To compare treatment responses, you want to examine *Asif* and *Afridi's* glucose levels at "Baseline" and "Treatment2".

Task: Extract this subset as a new matrix.

Problem 14: Extracting Row Subsets

Background: A researcher is focusing on *Harun's* results but only for the first two conditions.

Task: Extract these values.

Section C – Data Frames

Problem 16: Building a Patient Data Frame

Background: A clinical study records multiple patient attributes:

- Name: *Tamim*, *Imrul*, *Fiz*
- Age: 35, 39, 37
- Number of mutations detected: 4, 1, 0
- Married: TRUE, TRUE, TRUE

Such structured data is usually stored in a **data frame**.

Task: Create a data frame from this data.

Problem 17: Exploring Data Frame Structure

Background: Before analysis, you need to check the column types (numeric, logical, character) and overall structure.

Task: Use `str()` and `View()` to explore the data.

Problem 18: Renaming Columns

Background: Your PI (principal investigator) requested clearer column names: "Patient", "Age", "Mutations", "Married".

Task: Rename the columns accordingly.

Problem 19: Extracting a Subset of Data

Background: The epidemiologist needs to quickly review the "Name" and "Age" of the first two patients for age distribution.

Task: Use indexing to extract this sub-data frame.

Problem 20: Extracting a Single Column

Background: A geneticist wants only the "Mutations" column to calculate mutation frequency.

Task: Extract it using both `df[,3]` and `df$Mutations`.

Problem 21: Adding Row Names

Background: You want to add row names so you can easily reference patient-specific rows later without remembering row numbers.

Task: Assign "tamim", "imrul", "fiz" as row names.

Problem 22: Updating a Value

Background: A correction came in: *Tamim* actually had **5 mutations**, not 4. Data cleaning is a critical step before publishing results.

Task: Update this value in the data frame.

Problem 23: Sorting Patients by Age

Background: The PI wants the patients sorted by age in ascending order to quickly review the youngest to oldest.

Task: Sort the data frame by the "Age" column.

Section D – Factors

Problem 24: Academic Titles

Background: Your research team members hold titles: "lecturer", "assistant_professor", "associate_professor", "professor". In statistical modeling, factors preserve the hierarchy of such categories.

Task: Create a factor variable with levels ordered from lecturer to professor.

Problem 25: Lab Storage Temperature

Background: Samples in your biobank are labeled by storage conditions: "cold", "medium", "hot". By default, R treats this as unordered text, but in experiments temperature should be handled with defined order.

Task: Convert these into a factor with the order "medium" < "cold" < "hot".

Please try to do these questions. These will enhance your ability to think how our lecture in R links to biology.

Problem 1: Identifying Patients with Abnormal Glucose Patterns

Background:

You conducted an **Oral Glucose Tolerance Test (OGTT)** on 5 patients. Each patient has glucose values (mg/dL) at 3 time points:

- **Fasting, 1-hour, and 2-hours after glucose drink.**

Normal interpretation:

- Fasting glucose should be **<100**.
- At 1-hour, glucose rises but should not exceed **180**.
- At 2-hours, glucose should return to **<140**.

Here is the dataset:

Patient Fasting 1-hour 2-hour

P1	95	160	120
P2	110	190	170
P3	85	150	130
P4	98	200	180
P5	102	175	150

Task:

1. Create a **matrix** of the glucose values (rows = patients, columns = time points).
2. Identify which patients have **abnormal results at any time point**.
3. Return only the abnormal patients and their corresponding values.

Problem 2: Differential Gene Expression Analysis

Background:

In a small **RNA-seq pilot study**, you measured expression counts of 5 genes across 2 conditions:

- **Healthy tissue vs Tumor tissue.**

Gene Healthy Tumor

TP53	250	800
BRCA1	120	130

Gene Healthy Tumor

MYC	500	2000
EGFR	300	900
ACTB	1000	1000

A gene is considered **differentially expressed (DEG)** if the fold-change (Tumor ÷ Healthy) is:

- $\geq 2 \rightarrow$ Upregulated in tumor
- $\leq 0.5 \rightarrow$ Downregulated in tumor
- Otherwise $\rightarrow$ No change

Task:

1. Create a **data frame** with this data.
2. Compute the fold-change for each gene.
3. Add a new column "Status" labeling genes as "Upregulated", "Downregulated", or "No Change".
4. Which genes are upregulated in tumors?

Problem 3: Clinical Trial Stratification

Background:

In a clinical trial testing a new blood pressure drug, you enrolled 8 patients. You recorded:

- **Patient ID, Age, Baseline BP, Post-treatment BP.**

ID Age Baseline_BP Post_BP

P1	45	160	140
P2	50	150	135
P3	39	145	142
P4	60	170	155
P5	55	160	150
P6	42	135	130
P7	48	155	138
P8	52	162	145

A patient is considered **“Responder”** if their BP dropped by ≥ 15 units, otherwise **“Non-responder”**.

Task:

1. Create a **data frame** with this dataset.
2. Add a new column "Response" labeling each patient as "Responder" or "Non-responder".
3. Use indexing to extract only the **responders older than 50 years**.

Problem 4: Cancer Staging with Factors

Background:

You are analyzing data from a small cancer registry. Patients are classified into stages: "Stage I", "Stage II", "Stage III", "Stage IV".

Patient Stage

P1	II
P2	IV

Patient Stage

P3 I
P4 III
P5 II

Staging is **ordinal**: $I < II < III < IV$.

This ordering is crucial for statistical modeling (e.g., survival analysis).

Task:

1. Store the staging as a **factor with correct order**.
2. Count how many patients are in each stage.
3. Extract patients who are **Stage III or Stage IV** (advanced cancer).

Problem 5: Epidemiology – Identifying High-Risk Cities

Background:

An epidemiologist is studying COVID-19 cases across 6 cities. Data recorded:

- **City name**
- **Population (in millions)**
- **Cases (in thousands)**

City	Population_m	Cases_k
Dhaka	21	350
Chittagong	9	150
Rajshahi	3	40
Sylhet	2	70
Khulna	5	60
Barisal	1.5	15

Epidemiologists often compute **cases per 100,000 population** to compare risk across regions.

Task:

1. Create a **data frame** with the dataset.
2. Compute cases per 100,000 population for each city.
3. Add a new column "Risk_Level":
 - "High" if ≥ 200 per 100k
 - "Moderate" if 100–199
 - "Low" if < 100
4. Which cities fall into the **High Risk** category?